

CS336 Ray Tracer Report

By: Tyler Schaefer

Introduction

This report details the decision-making and programming that went into creating a ray tracer for CS336. To be succinct, the report provides only a high-level overview of the program. The source code is available on my [GitHub](#) for a more detailed look.

What is a Ray Tracer?

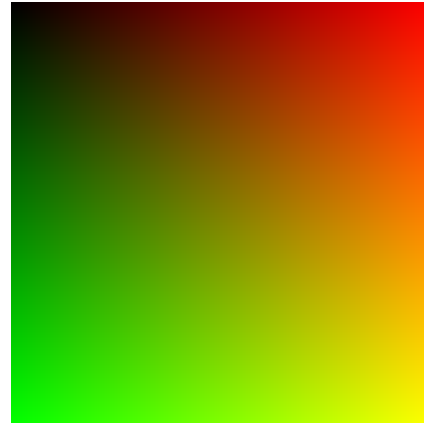
A ray tracer simulates light bouncing around in a virtual environment and attempts to produce a realistic representation of that environment. This project is technically a path tracer because the simulated light rays can bounce multiple times.

A pixel within the final image is designated a certain color based on the ray(s) that would have gone through the virtual viewer, that pixel, traversed the scene, and come from a light source. Because most rays from a light source never reach the viewer, we can greatly reduce the number of light rays necessary by having the rays originate at the “eye” or “camera” (rather than light sources) and propagate out into the scene. These rays keep track of what colors they reflect or scatter off of and get illuminated if they reach a light source.

First Steps

To build a solid base, linear algebra is essential. All of the 3-dimensional geometry is best modeled with points and vectors. Which in turn is manipulated with linear algebra. To begin, I wrote a simple vector and point module.

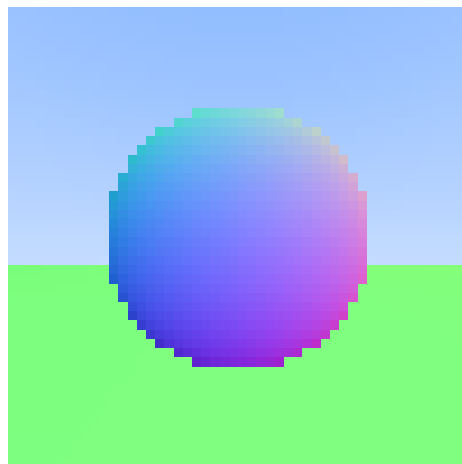
Another fundamental feature is to be able to output images. For this, I used a Portable Pixel Map (PPM). These are simple to write and can be written in either plaintext or binary. PPMs are a lossless, uncompressed file format; while this makes them rather large files, they are also simple for debugging.



(Color gradient test)

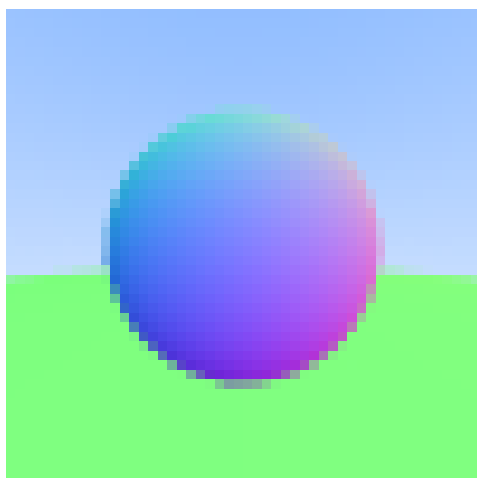
Camera

After writing those basic fundamentals, my next step was setting up the ray tracing algorithm. First, I extended the vector module to have rays (a point and a vector). Then, I implemented the recursive tracing in the camera module. The camera module simulates everything necessary to set up the viewing angle and actually render the scene. To render, each ray begins at the camera and is given a direction to go through the center of a pixel. The ray then recursively bounces as it reaches different objects within the scene (To render these scenes, spheres are used, and geometry is covered later).



(50x50 basic render)

However, the aliasing is quite bad in this image. To improve this, multiple rays are sent per pixel (each randomized to a slightly different direction), and the pixel is given the averaged color of all the rays through it.



(Same scene with anti-aliasing)

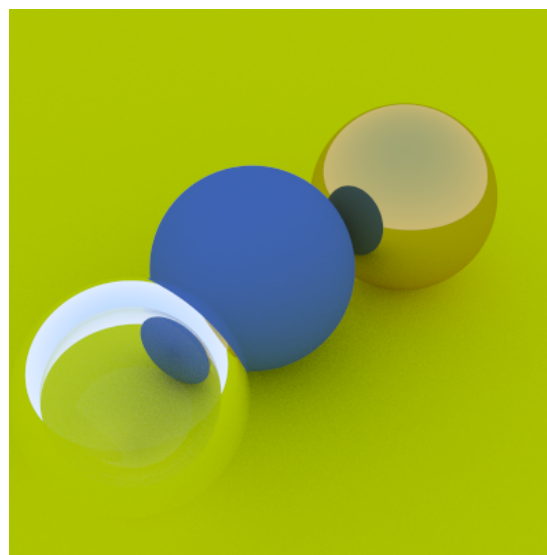
Finally, to improve our scene-building potential, we want our camera to be able to be placed anywhere and face in any direction. This is done with a cross-product to define a basis for the camera and the direction it is looking. And adjust the rays to go in the proper direction. The camera is also initialized with an aspect ratio, the number of pixels in the image, and the FOV.

Materials

It is common to separate materials and textures from the geometry within almost all ray tracers. The simplest material to implement is Lambertian, a plain, matte surface. This scatters light in a normal distribution on the hemisphere about where the light ray strikes the material.

The next material I implemented was reflective surfaces, like metals. For these, the light reflects perfectly, making it a simple calculation. I also added a roughness to the material so that the reflection gets progressively “fuzzier.”

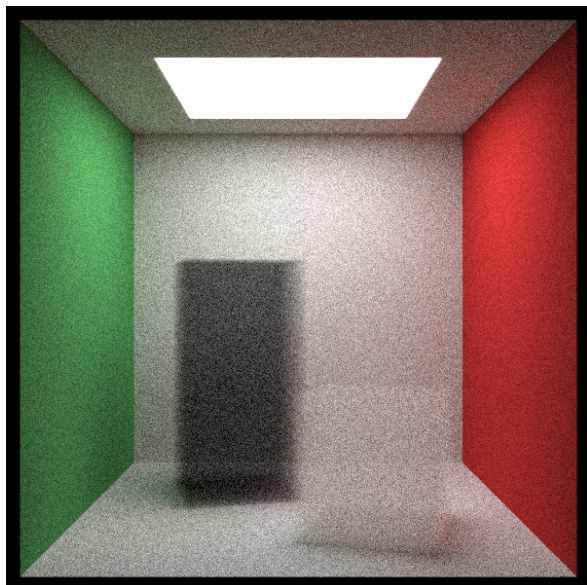
The third material in the program is dielectric, clear materials. These are the most difficult as sometimes the material reflects and other times it refracts the light. To do this cheaply, I used Schlick’s approximation, which uses the angle at which the light strikes the material to decide if it should reflect or refract.



(Spheres with various materials)

The fourth material I implemented is isotropic, also referred to as constant mediums, i.e., fog and smoke. To implement this I made it so that rays of light can penetrate the geometry of an object, and there is a randomization to when/where they reflect. This can be tuned to provide thicker and thinner constant mediums.

The final but most important material is diffused lighting. This is almost identical to Lambertian; however, the light's albedo (base color) can be larger than 1. Also, if a light is struck, the ray doesn’t continue to bounce, searching for further sources; the light simply is that color. This allows it to illuminate other objects and has to be capped before outputting because PPMs cannot support color values above 255 bits.

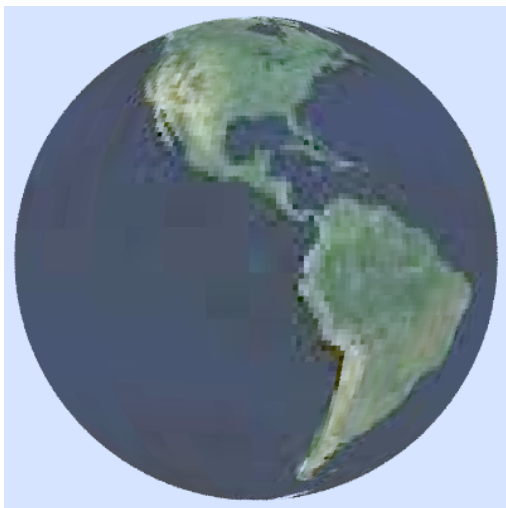


(Cornell box with Isotropic cubes and diffuse lighting)

Texturing

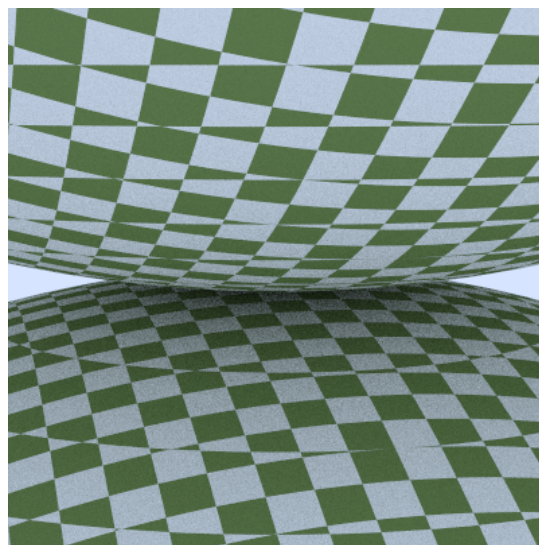
While a material describes how light interacts with objects, i.e., scattering, reflecting, and refracting, a texture describes the pattern of an object. These allow for more interesting objects that aren't a single color.

The first type of texture in the program is to load an image with the [stb_image.h](#) header. The image can then be applied to any shape.



(Image Texture)

Another texture is the checkered pattern, which uses 2 sub-textures and applies them alternating in a grid. This texture isn't applied to the u-v space of geometry but rather to the global coordinate space.



(Checker Texture)

The final texture currently implemented is Perlin texturing. This produces a marbling effect with random noise, turbulence, and sine waves. The values can be tweaked to get drastically differing effects.

Geometry

Geometry is fundamental to building scenes within a ray tracer. The simplest geometry to build is a sphere. Just a point and a radius. I can then solve the equation for the sphere in 3D set equal to the ray's equation to find if there is ever a time the ray intersects the sphere.

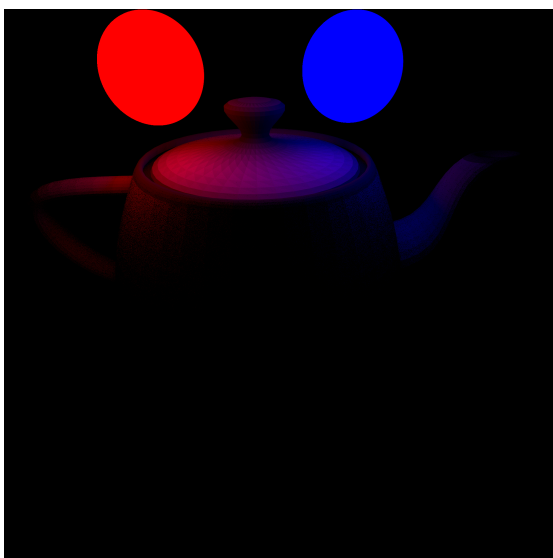
The next geometry form I implemented was a rectangular quad. These are defined with a point and 2 vectors that depict a portion of a plane. To detect collisions, I find where the ray intersects the plane (if it does) and then check if that is within the quadrilateral.

Rectangular prisms were a simple extension from quadrilaterals. I can define the points of the prism and then simply have the prism be a list of quadrilaterals connecting those points.

The final geometry in the program, but one of the most essential, is the triangle. These are implemented using barycentric coordinates. First, I find if the ray intersects the plane and then get its intersection point's barycentric coordinates relative to the triangle. If the point lies within the triangle, all coordinates will be positive. This allows me to detect collisions.

Loading Meshes

Now that I have triangles I can build far more complex geometry that are meshes of lots of triangles. To load these meshes, I used the [tinyobjloader_c.h](#) header. The obj files define the vertices of points within the mesh and which points are connected by a triangle. The obj loader handles all of that for me and provides a list of triangles I need to build the mesh.



(Teapot mesh ~6000 triangles)

Acceleration Structure

Now that I have many more shapes to check for collisions, optimizing the search for collisions is important. I did this with a Bounding Volume Hierarchy (BVH). This is a tree of boxes that covers the geometry. If a box is hit, its' sub-boxes are checked. This way, the tree can be recursively checked down until a collision is found. This takes looking for collisions from $O(n)$ to $O(\sqrt{n})$ (Plus some overhead for checking box collisions).

To make the boxes cheap to build, store, and check, I used Axis-Aligned Bounding Boxes, which always sit square to the camera and can then be defined with two points. To build the BVH, I also sort the objects along the longest axis. This builds a more efficient structure for searching.

Improving Performance

The BVH improves support for dense scenes, but large renders are still quite computationally expensive. To combat this, I implemented several features focused on improving the efficiency and performance of the program.

The first of these features was to parallelize the program. This was rather simple as when rendering, each pixel is completely independent of the others, and the scene is read-only. The only synchronization necessary is finding what to work on next and writing the rendering results out.

This parallelization is done within the camera module using the pthreads library. I have each thread take an entire line of pixels to render in each batch to reduce the overhead from trying to over-synchronize the threads. This approach also avoids the other extreme of completely chunking the scene to reduce the chance that a single thread receives a far harder portion to render than any other thread.

The next optimization feature I added was object instancing. This allowed for dense

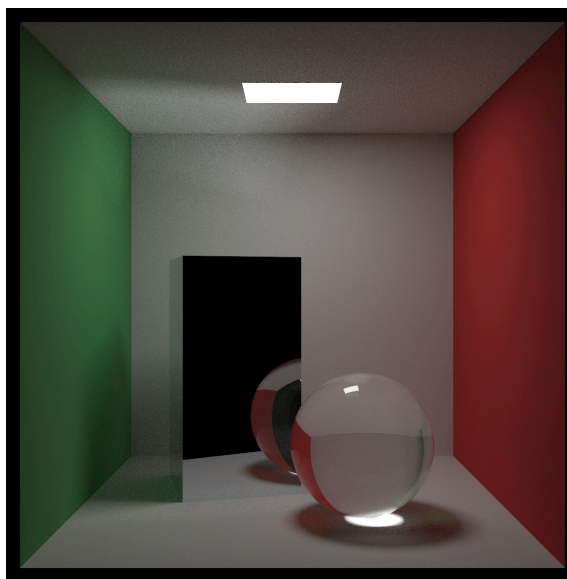
scenes with repeated geometry to use far less memory. This is done using the same geometry in multiple places but “instancing” it to rotate, translate, and scale it within the scene. The rays receive the inverse of these operations to check collisions and strike the same object even if it is instanced multiple times. This is also extremely helpful for manipulating the meshes of triangles within a scene rather than having to edit the obj file.

Another optimization was to use adaptive sampling with the light rays. As we send out lots of rays per pixel (thousands for a high-quality render), some of the rays are redundant. To fix this, I used the illuminance of the color returned by each ray to track the mean illuminance and variance in illuminance for the pixel. Then, I will occasionally check these for the pixel, and if, after enough samples, the illuminance is within a 95% confidence interval, I stop sampling early. This drastically reduces the number of rays needing to be sent to simple areas of the scene, i.e., backgrounds and matte objects. However, it cannot improve performance where the scene is complex.

That is where importance sampling comes in. Importance sampling is adjusting where a bouncing/scattered ray goes, sending it to more “important” parts of the scene, i.e., lights and complex areas. This is trying to reduce the number of rays necessary to approach the correct color for a pixel. The adjustment by itself, though, would ruin the accuracy of the render. To correct this, the color returned from an adjusted ray is scaled by a probability density function (pdf) relative to how likely the ray was to bounce in the adjusted direction. This is calculated as the portion of the area of the hemisphere that the ray could bounce into that is covered by a projection of the important object.

To do this in practice can be somewhat expensive so it is done with a Monte Carlo simulation. Rather than being 100% mathematically correct at every stage of the

rendering (which would require calculating complex integrals for each ray), the Monte Carlo simulation guarantees that, given more samples, the simulation will approach the correct output. This greatly reduces the number of rays necessary to get near the correct color.



(Cornell box with importance sampling)

Render Quality

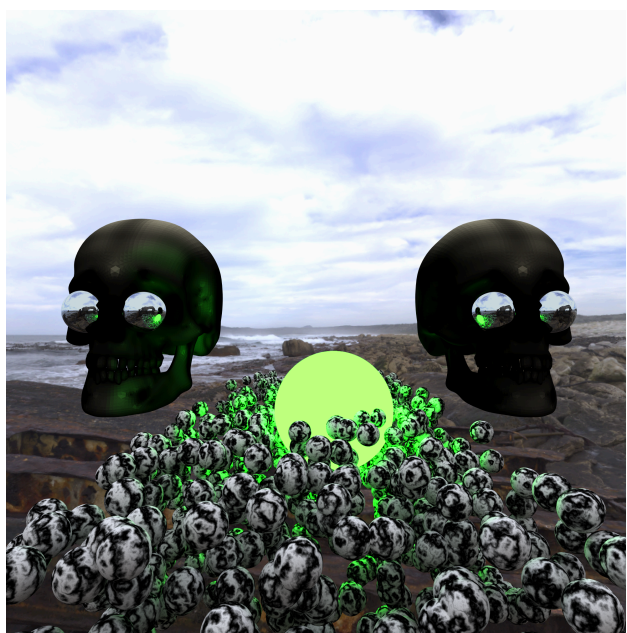
After rendering was optimized I focused on improving the quality of images produced. The first step towards that was adding depth of field. This is done by adding a “lens” to our virtual camera to focus the rays on a certain plane. Causing the blurring at other distances from the camera.

Another change was to add stratified sampling. To do this, rather than picking my rays randomized about the center of the pixel, I subdivided the pixel into an 8x8 grid and randomized the rays through the center of the smaller grid. This ensures that the pixel is more properly covered in samples without biasing the samples. This helps provide sharper edges to objects.

An easy yet striking visual change was to add sky boxes. These are imported images

(same as textures) that are referenced for the background color whenever a ray misses all geometry. This is done by treating the sky box as infinitely far away on a sphere surrounding the scene and using the direction the ray went in when it missed.

The final change for render quality was to support High Dynamic Range (HDR) images. This allows for HDR sky boxes to provide illuminance for the scene. I also changed the output file format to HDR rather than PPM.



(Render with HDR sky-box)

Berkeley Computer Science, *Adaptive Sampling*,
<https://cs184.eecs.berkeley.edu/sp24/docs/hw3-1-part-5>

Sean Barrett, *Image Loading Header*,
 stb_image.h,
https://github.com/nothings/stb/blob/master/stb_image.h

Syoyo Fujita, *Tiny Obj File Loader*,
 tinyobjloader.c.h,
https://github.com/syoyo/tinyobjloader-c/blob/master/tinyobj_loader_c.h

Sources

Peter Shirley, Trevor D. Black, and Steve Hollasch, *Ray Tracing in One Weekend: The Book Series*, <https://raytracing.github.io/>

Matt Pharr, Wenzel Jakob, and Greg Humphreys, *Physically Based Rendering: From Theory to Implementation*, 4th edition, ISBN 0262048027, available for free online at <https://pbr-book.org/>